Operating Systems Lab

Practice Questions & Complete Solutions

Lab Assignments - All 5 Sets

Contents

Set 1 **1**

Set 2 **4**

Set 3 **6**

Set 4 **8**

Set 5 **11**

Set 1

Question 1

Create the directory structure `os_practical/{input, output, scripts, archive}` using a single command. (2 Marks)

Answer:

```
mkdir -p os_practical/{input,output,scripts,archive}
```

Question 2

Inside `scripts`, create `run.sh`. Set permissions so that the owner can read, write, and execute the file, while the group and others can only read it. Verify the permissions using `ls -l`. (3 Marks)

Answer:

```
touch os_practical/scripts/run.sh
chmod 744 os_practical/scripts/run.sh
ls -l os_practical/scripts/run.sh
```

Question 3

Create and implement the following Python program:

- Create `scripts/process_demo.py` using `os.fork()`, and handle fork failure.
- The child must print its PID and parent PID, then terminate with exit status 5.
- The parent must print its PID and child PID, wait for the child, and display the child's exit status. (6 Marks)

Answer:

```
import os

rs = os.fork()

if rs == 0:
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
    os._exit(5)
elif rs > 0:
    print("Parent PID:", os.getpid())
    print("Child PID:", rs)
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Question 4

Write a Python program using `os.fork()`, `os.exec*()`, and `os.waitpid()` to:

i. Display the current directory and the contents of the archive directory in long format.

ii. Display disk-space usage in human-readable format and system uptime.

iii. Display the current date and time.

Use one child per command, wait for completion before continuing, and handle fork/exec errors. (9 Marks)

Answer:

```
import os

# 1. pwd
rs = os.fork()
if rs == 0:
    os.execvp("pwd", ["pwd"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 2. ls -l os_practical/archive
rs = os.fork()
if rs == 0:
    os.execvp("ls", ["ls", "-l", "os_practical/archive"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
```

```
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 3. df -h
rs = os.fork()
if rs == 0:
    os.execvp("df", ["df", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 4. uptime
rs = os.fork()
if rs == 0:
    os.execvp("uptime", ["uptime"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 5. date
rs = os.fork()
if rs == 0:
    os.execvp("date", ["date"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Set 2

Question 1

Create the directory structure `systemlab/{data, logs, results, backup}` using a single command. (2 Marks)

Answer:

```
mkdir -p systemlab/{data,logs,results,backup}
```

Question 2

Create `logs/system.log`. Set its permissions to read/write for the owner, read-only for the group, and no permissions for others. Display the final permissions using `ls -l`. (3 Marks)

Answer:

```
touch systemlab/logs/system.log
chmod 640 systemlab/logs/system.log
ls -l systemlab/logs/system.log
```

Question 3

Create and implement the following Python program:

- Create `data/parent_child.py` using `os.fork()`, including error handling if fork fails.
- The child prints "Child running" with its PID and parent PID, then exits with status 3.
- The parent prints its PID and the child PID, waits for the child, and reports the exit status. (6 Marks)

Answer:

```
import os

rs = os.fork()

if rs == 0:
    print("Child running")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
    os._exit(3)
elif rs > 0:
    print("Parent PID:", os.getpid())
    print("Child PID:", rs)
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Question 4

Write a Python program using `os.fork()`, an `os.exec*()` function, and `os.wait()` or `os.waitpid()` to:

i. Show the current working directory and list files, including hidden files, in long format.

ii. Display memory usage and the current user's running processes.

iii. Display the number of seconds the system has been running (uptime).

Create a child for each command, wait for each child sequentially, and show an error message if a command cannot be executed. (9 Marks)

Answer:

```python
import os

# 1. pwd
rs = os.fork()
if rs == 0:
    os.execvp("pwd", ["pwd"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 2. ls -la
rs = os.fork()
if rs == 0:
    os.execvp("ls", ["ls", "-la"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 3. free -h
rs = os.fork()
if rs == 0:
    os.execvp("free", ["free", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 4. ps -u $USER
rs = os.fork()
if rs == 0:
    os.execvp("ps", ["ps", "-u", os.getenv("USER")])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 5. uptime
rs = os.fork()
if rs == 0:
    os.execvp("uptime", ["uptime"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Set 3

Question 1

Create the directory structure `linux_exam/{documents, temporary, programs, logs}` using a single command. (2 Marks)

Answer:

```
mkdir -p linux_exam/{documents,temporary,programs,logs}
```

Question 2

Create `documents/notes.txt`. Set permissions so the owner has read/write access, the group has read-only access, and others have read-only access. Verify using `ls -l`. (3 Marks)

Answer:

```
touch linux_exam/documents/notes.txt
chmod 644 linux_exam/documents/notes.txt
ls -l linux_exam/documents/notes.txt
```

Question 3

Create and implement the following Python program:

- Create `programs/fork_wait.py`. Use `os.fork()` and handle a possible fork error.
- The child prints its PID and parent PID, then exits with status 7.
- The parent prints its PID and child PID, waits for the child, and prints the decoded exit status. (6 Marks)

Answer:

```
import os

rs = os.fork()

if rs == 0:
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
    os._exit(7)
elif rs > 0:
    print("Parent PID:", os.getpid())
    print("Child PID:", rs)
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Question 4

Write a Python program using `os.fork()`, `os.exec*()`, and `os.waitpid()` to:

i. Print the current working directory and list the contents of documents in detailed format.

ii. Display disk usage for mounted file systems and memory usage in human-readable format.

iii. Display the current date and time.

Run each command in a separate child process, wait for it to finish before continuing, and handle command-execution failures. (9 Marks)

Answer:

```
import os

# 1. pwd
rs = os.fork()
if rs == 0:
    os.execvp("pwd", ["pwd"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 2. ls -l linux_exam/documents
rs = os.fork()
if rs == 0:
    os.execvp("ls", ["ls", "-l", "linux_exam/documents"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 3. df -h
rs = os.fork()
if rs == 0:
    os.execvp("df", ["df", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 4. free -h
rs = os.fork()
if rs == 0:
    os.execvp("free", ["free", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 5. date
rs = os.fork()
if rs == 0:
    os.execvp("date", ["date"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Set 4

Question 1

Create the directory structure `labwork/{source, backup, output, monitoring}` using a single command. (2 Marks)

Answer:

```
mkdir -p labwork/{source,backup,output,monitoring}
```

Question 2

Inside `monitoring`, create `monitor.log`. Set permissions to `rw-r-----` (owner read/write, group read-only, others no access) and show the result using `ls -l`. (3 Marks)

Answer:

```
touch labwork/monitoring/monitor.log
chmod 640 labwork/monitoring/monitor.log
ls -l labwork/monitoring/monitor.log
```

Question 3

Create and implement the following Python program:

- Create `source/fork_status.py` using `os.fork()`, and handle the case where fork fails.
- The child prints ”Child process started” with its PID and parent PID, then exits with status 2.
- The parent prints its PID and the child PID, waits for the child, and displays the exit status. (6 Marks)

Answer:

```
import os

rs = os.fork()

if rs == 0:
    print("Child process started")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
    os._exit(2)
elif rs > 0:
    print("Parent PID:", os.getpid())
    print("Child PID:", rs)
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Question 4

Write a Python program that uses `os.fork()`, an `os.exec*()` function, and `os.wait()` or `os.waitpid()` to:

i. Display the present working directory and list all entries, including hidden files, in long format.

ii. Display system uptime and disk-space usage in human-readable format.

iii. Display processes belonging to the current user and the current date/time.

Use a separate child process for each command, wait for each child sequentially, and handle fork/exec errors. (9 Marks)

Answer:

```
import os

# 1. pwd
rs = os.fork()
if rs == 0:
    os.execvp("pwd", ["pwd"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 2. ls -la
rs = os.fork()
if rs == 0:
    os.execvp("ls", ["ls", "-la"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 3. uptime
rs = os.fork()
if rs == 0:
    os.execvp("uptime", ["uptime"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 4. df -h
rs = os.fork()
if rs == 0:
    os.execvp("df", ["df", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 5. ps -u $USER
rs = os.fork()
if rs == 0:
    os.execvp("ps", ["ps", "-u", os.getenv("USER")])
```

```
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 6. date
rs = os.fork()
if rs == 0:
    os.execvp("date", ["date"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Set 5

Question 1

Create the directory structure `final_lab/{reports, logs, programs, backup}` using a single command. (2 Marks)

Answer:

```
mkdir -p final_lab/{reports,logs,programs,backup}
```

Question 2

Create `reports/final.txt`. Change its permissions so that the owner has read/write/execute access, the group has read access, and others have no access. Verify using `ls -l`. (3 Marks)

Answer:

```
touch final_lab/reports/final.txt
chmod 740 final_lab/reports/final.txt
ls -l final_lab/reports/final.txt
```

Question 3

Create and implement the following Python program:

- Create `programs/child_info.py` using `os.fork()`, including handling for fork failure.
- The child prints its PID and parent PID and exits with status 4.
- The parent prints its PID and child PID, waits for the child, and displays the child's exit status. (6 Marks)

Answer:

```
import os

rs = os.fork()

if rs == 0:
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
    os._exit(4)
elif rs > 0:
    print("Parent PID:", os.getpid())
    print("Child PID:", rs)
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```

Question 4

Write a Python program using `os.fork()`, an appropriate `os.exec*()` function, and `os.wait()` or `os.waitpid()` to:

i. Display the current working directory and list all files, including hidden files, with detailed information.

ii. Display memory and disk-space usage in human-readable format.

iii. Display current-user processes and the current date/time.

Create a separate child process for each command, wait for completion before proceeding, and display suitable error messages for failed command execution. (9 Marks)

Answer:

```
import os

# 1. pwd
rs = os.fork()
if rs == 0:
    os.execvp("pwd", ["pwd"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 2. ls -la
rs = os.fork()
if rs == 0:
    os.execvp("ls", ["ls", "-la"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 3. free -h
rs = os.fork()
if rs == 0:
    os.execvp("free", ["free", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 4. df -h
rs = os.fork()
if rs == 0:
    os.execvp("df", ["df", "-h"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 5. ps -u $USER
rs = os.fork()
if rs == 0:
    os.execvp("ps", ["ps", "-u", os.getenv("USER")])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")

# 6. date
rs = os.fork()
if rs == 0:
```

```
    os.execvp("date", ["date"])
elif rs > 0:
    pid, status = os.waitpid(rs, 0)
    print("Child Exit Status:", os.WEXITSTATUS(status))
else:
    print("Fork failed")
```